

PRODUCT REQUIREMENTS DOCUMENT

VoiceCircle

AI that protects seniors from voice scams using trusted family voices, practice calls, deepfake checks and daily check-ins.

Version	2.0 – FastAPI build edition
Date	October 8, 2026
Event	ForgeHacks Online 2026 (AI + Cybersecurity primary, AI + Healthcare secondary)
Backend	Python 3.11+, FastAPI, Supabase (Postgres, Auth, Storage)
Frontend	React + TypeScript, built with Claude Code against the API contract in Section 7
Voice / Telephony	Telnyx (calls, SMS, AI assistant), Resemble AI (voice identity, cloning, detection)
Owner	Henry Tech

Deadline: Submissions close **October 10, 2026 at 12:00 PM EDT** (5:00 PM WAT, Lagos time)¹. The event runs October 3–10, 2026, fully online². This PRD assumes building starts October 8 and is scoped so a solo developer, or a small team, can finish a demo-ready MVP in about 48 hours.

Contents

- 1. Product overview
- 2. Users and personas
- 3. Feature requirements
- 4. System architecture
- 5. Backend project structure
- 6. Data model
- 7. API contract (frontend to backend)
- 8. Call flows and webhooks
- 9. Frontend spec for Claude Code
- 10. Security, privacy and ethics
- 11. Build plan (48 hours)
- 12. Demo script and submission
- 13. Risks and open questions

¹ ForgeHacks Devpost, dates page: <https://forgehacks-2026.devpost.com/details/dates>

² ForgeHacks official site: <https://www.forgehacks.dev/>

1. Product overview

Problem

AI voice cloning now lets scammers sound exactly like a grandchild or a daughter. Older adults are the main target of “family emergency” scams (“Grandma, I’m in trouble, send money now”). Most protection tools assume the person will install an app and understand words like “deepfake”. Many seniors won’t.

Solution

VoiceCircle works over ordinary phone calls and texts. Family sets it up on the web; the senior only ever answers the phone. It has four pillars:

Pillar	What it does	Who uses it
Voice Circle	A trusted contact records a short voice sample once. It becomes a verified voice reference.	Family
Scam Training	VoiceCircle calls the senior with a practice scam in the cloned voice of a trusted contact, at Easy / Medium / Hard difficulty, and scores the reaction.	Senior (on the phone), family (results)
Deepfake Check	Upload a suspicious call recording and get a plain-language REAL / FAKE result against the verified voice.	Family or senior
Companion Calls	“Hugh”, a friendly AI, calls daily, chats, and tracks speech and memory signals over time. Family is alerted if something changes.	Senior (on the phone), family (trends, alerts)

Goals for the hackathon

- One working end-to-end demo: enroll a voice, run a practice call, check a recording, show a companion call with a trend.
- Every feature runs live against real APIs (no mocked call flow in the demo).
- Clean split: FastAPI owns all business logic and third-party calls; the frontend only talks to FastAPI and Supabase Auth.
- A 2–4 minute demo video, a public GitHub repo with a clear README, and a Devpost write-up.

Non-goals

- Native mobile apps.
- Billing, subscriptions, multi-tenant admin.
- Clinical-grade cognitive assessment. Companion metrics are wellness signals, not diagnoses.
- Production-scale call infrastructure or compliance certification.

2. Users and personas

Persona	Context	Needs
Senior (primary)	65+, uses a basic mobile or landline, may have vision or hearing limits, not tech-savvy.	Zero setup. Warm, simple language. Practice without real risk. A clear yes/no on whether a voice is real.
Trusted contact (secondary)	Adult child, grandchild or caregiver. Comfortable with web apps.	Fast onboarding, voice enrollment in under 2 minutes, visibility into practice results and alerts.
Judge / demo viewer	Watches a 2–4 minute video.	Understands the problem in 15 seconds and sees each pillar work live.

3. Feature requirements

Priority key: **P0** = must ship for demo, **P1** = ship if time allows, **P2** = post-hackathon.

3.1 Voice Circle – trusted voice enrollment

User flow

- Family member signs up (Supabase Auth, email magic link or password).
- Creates a Circle (e.g. “Grandma Ada's Circle”) and adds the senior: name, phone (E.164), preferred call time, timezone.
- Records a voice sample in the browser by reading 3–5 on-screen prompts (about 30–60 seconds total), or uploads a file.
- Ticks an explicit consent box: “I agree my voice can be used to verify calls and to create practice calls for this Circle only.”
- FastAPI uploads the audio to Supabase Storage, sends it to Resemble AI to create the voice identity and clone, and saves the returned IDs.
- UI shows status: Not enrolled / Processing / Enrolled / Failed.

Requirements

ID	Requirement	Priority
VC-1	Create Circle, add one senior and at least one trusted contact.	P0
VC-2	Record audio in browser (MediaRecorder, webm/opus) or upload wav/mp3/m4a, max 20 MB.	P0
VC-3	Consent checkbox required; consent timestamp and text version stored.	P0
VC-4	Backend creates Resemble voice identity + clone; stores provider IDs on the member row.	P0
VC-5	Enrollment status polled by frontend (or Supabase Realtime).	P0
VC-6	Multiple trusted contacts per Circle; delete my voice (removes provider voice + files).	P1

Acceptance: a trusted contact enrolls in under 2 minutes, sees “Enrolled”, and the voice can be used by Scam Training and Deepfake Check.

3.2 Scam Training – practice scam calls

User flow

- Family opens Training, picks the senior, the voice to clone, a scenario and a difficulty, then clicks “Call now” (or schedules it).
- FastAPI places an outbound call with Telnyx Call Control.
- On answer, the cloned voice plays the scenario. The call listens to the senior's replies (Telnyx transcription), and an LLM picks the next line in character.
- The call ends when the senior hangs up, refuses, asks a verification question, or “complies” (says they will send money or reads out details).
- Within 15 seconds of hang-up, a short debrief SMS or voice message goes to the senior: “This was a VoiceCircle practice call. You did great by...”
- Family sees the outcome, score and transcript on the dashboard.

Difficulty design

Level	Script style	Red flags present
Easy	Pre-generated audio clips, fixed script. Generic greeting, obvious urgency.	Unknown number, “don't tell Mom”, gift cards, strange phrasing.
Medium	LLM-driven, uses the contact's first name and the senior's name.	Urgency and secrecy, but natural language. Wire transfer request.
Hard	LLM-driven, adds one true personal detail family provided (e.g. pet name, city), handles pushback.	Subtle pressure, plausible story (car accident, bail, stuck abroad).

Outcome scoring

Outcome	Detected when	Score
hung_up_early	Senior hangs up within 30 s of the ask	100
verified	Senior asks a verification question or says they'll call back on a known number	90
refused	Senior clearly says no	80
hesitated	Senior asks questions but keeps engaging past the money ask	50
complied	Senior agrees to pay, or reads out account, card or address details	0

Classification is done by the LLM on the final transcript, returning strict JSON. Keep a keyword fallback (“gift card”, “wire”, “I'll send”) in case the LLM call fails.

Requirements

ID	Requirement	Priority
ST-1	Place a real outbound call through Telnyx and play cloned-voice audio.	P0
ST-2	At least Easy and Hard difficulty working end to end.	P0
ST-3	Store transcript, outcome, score, duration per call.	P0
ST-4	Debrief SMS to the senior after every practice call.	P0
ST-5	Hard safety rule: the script never asks for real card numbers; if the senior starts reading digits, the call interrupts with the debrief.	P0
ST-6	Scheduling (“practice call sometime this week”) and Medium level.	P1

3.3 Deepfake Check – real or fake

User flow

- User opens “Check a call”, picks who the caller claimed to be, uploads a recording.
- FastAPI stores the file, then runs two checks in parallel: (a) Resemble deepfake detection (is this synthetic speech?) and (b) speaker verification against the enrolled voice (is this the same person?).
- Results combine into one verdict with a plain-language explanation.

Verdict logic

Synthetic score	Speaker match	Verdict shown
Low	High	Looks REAL – this sounds like Alex.
High	Any	Looks FAKE – this voice was likely made by a computer.
Low	Low	NOT Alex – this is a real person, but not the person they claimed to be.
Unclear	Unclear	Not sure – please call Alex back on the number you know.

Thresholds live in config (DETECT_FAKE_THRESHOLD, SPEAKER_MATCH_THRESHOLD) so you can tune them on demo clips. Always show the safe action (“Hang up and call back”) regardless of verdict.

Acceptance: a 10–60 second clip returns a verdict in under 20 seconds; a cloned clip from Scam Training is flagged FAKE in the demo.

3.4 Companion Calls – daily check-ins with Hugh

User flow

- A scheduler in FastAPI places a call to each senior at their chosen local time.
- Hugh greets them by name, chats for 2–5 minutes (day, meals, family, hobbies), and asks one gentle recall question that references yesterday's call (“Yesterday you said you were baking. How did it turn out?”).
- After the call, FastAPI saves the transcript, computes metrics, compares them to the senior's baseline, and raises an alert if needed.
- Family sees call history, a trend chart and any alerts.

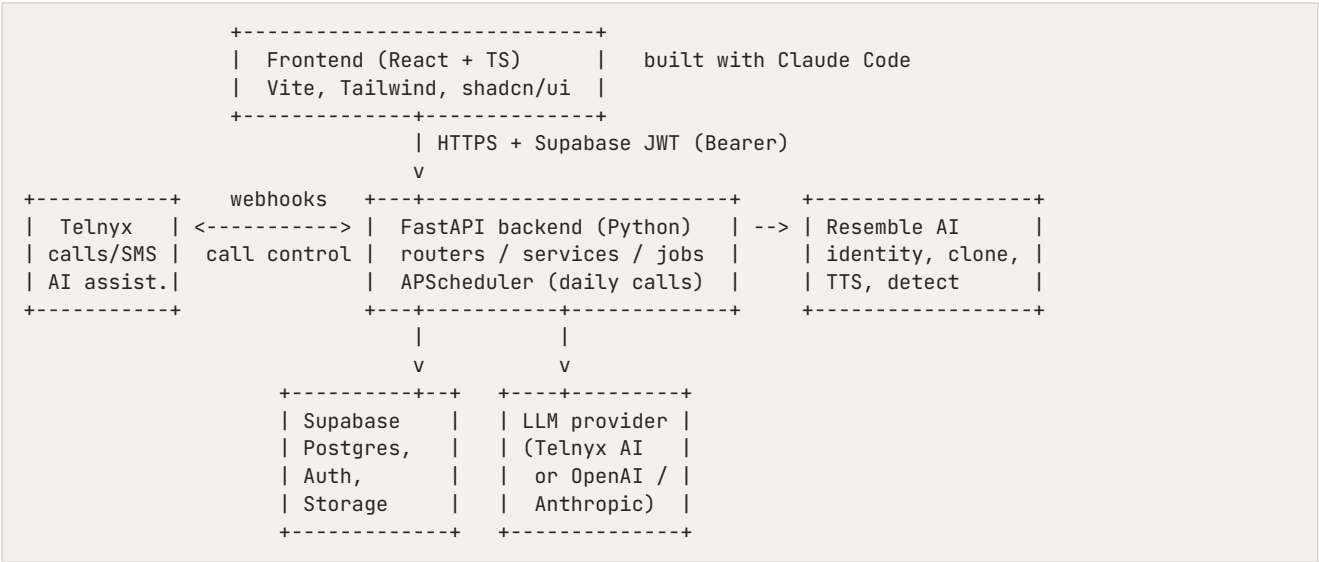
Metrics (MVP heuristics)

Metric	How to compute	Flag when
Words per minute	Senior's word count / senior's speaking time (from transcript timestamps)	< 75% of 7-day baseline
Average response latency	Gap between end of Hugh's turn and start of senior's reply	> 150% of baseline
Filler / pause rate	Count of um, uh, long pauses (>2 s) per 100 words	> 150% of baseline
Recall success	LLM grades the recall answer: correct / partial / none, against stored memory	2 “none” in a row
Repetition	Same story or question repeated within one call (LLM check)	Present 2 days in a row
Mood	LLM label: positive / neutral / low	“low” 3 days in a row

Baseline = rolling mean of the first 3–7 calls. For the demo, seed 7 days of companion history so the trend chart and an alert are visible.

Language matters: alerts must say “Hugh noticed Ada paused more than usual this week. You may want to check in.” Never “signs of dementia”. VoiceCircle is not a medical device.

4. System architecture



Responsibilities

Layer	Owns	Does not do
Frontend	UI, audio recording, auth session (Supabase JS client for login only), calling FastAPI, polling status.	No direct calls to Telnyx, Resemble or the LLM. No DB writes except through FastAPI.
FastAPI	Auth verification, business logic, all third-party calls, webhooks, scheduling, metrics, alerts.	No HTML rendering.
Supabase	Postgres, Auth (JWT issuer), Storage buckets (private).	Business logic (keep RLS as a safety net).
Telnyx	Outbound calls, audio playback, live transcription, SMS, optional AI Assistant for Hugh.	
Resemble AI	Voice identity, voice clone, speech synthesis in the cloned voice, deepfake detection.	

Key technical decisions

- **Cloned speech delivery:** generate audio with Resemble, upload to Supabase Storage, play it into the call via Telnyx `playback_start` with a signed URL. Simple and reliable for a hackathon.
- **Turn-taking for Scam Training:** Telnyx transcription webhooks deliver the senior's speech; FastAPI calls the LLM, synthesizes the next line, and plays it. Pre-generate Easy clips and the opening line of every scenario to hide latency.
- **Hugh:** use a Telnyx AI Assistant for the live conversation (lowest latency, least code). FastAPI injects memory (yesterday's summary) into the assistant instructions before dialing and pulls the transcript after hang-up. Fallback: run the same turn loop used by Scam Training with a standard TTS voice.
- **Background work:** FastAPI BackgroundTasks for short jobs (enrollment, detection), APScheduler for the daily companion schedule. No Celery needed for MVP.
- **Real-time UI:** polling every 2–3 s on call/enrollment status is enough. Supabase Realtime is a P1 upgrade.

Endpoint names for Telnyx and Resemble change over time; confirm exact paths and payloads in each provider's current docs before coding the service wrappers. Keep every provider call behind one service module so swapping is a one-file change.

5. Backend project structure

```
voicecircle-api/
  app/
    main.py          # FastAPI app, CORS, routers, scheduler startup
    config.py        # pydantic-settings: env vars, thresholds
    deps.py          # get_current_user (verify Supabase JWT), get_db
    db.py            # Supabase client (service role) or SQLAlchemy async engine
    models/          # Pydantic request/response schemas
      circle.py      member.py  practice.py  detection.py  companion.py  alert.py
    routers/
      auth.py        # /me
      circles.py     # circles + members
      voices.py      # enrollment
      practice.py    # scam training
      detection.py   # deepfake check
      companion.py   # companion calls, metrics
      alerts.py
      webhooks.py    # /webhooks/telnyx (call + sms events)
      demo.py        # seed + reset demo data (guarded by DEMO_MODE)
    services/
      telnyx_service.py # dial, play, speak, transcription, hangup, sms
      resemble_service.py # create identity/clone, synthesize, detect, verify
      llm_service.py   # scam turns, outcome classify, recall grading, summaries
      storage_service.py # upload, signed URLs
      metrics_service.py # wpm, latency, fillers, baseline compare
      alert_service.py  # create alert + notify family (SMS/email)
      call_state.py     # in-memory + DB state machine per active call
    prompts/
      scam_easy.yaml  scam_hard.yaml  hugh_system.md  classify_outcome.md
    jobs/
      scheduler.py    # APScheduler: companion calls, scheduled practice
    migrations/
      001_init.sql
    tests/
      .env.example
      requirements.txt
      Dockerfile
      README.md
```

Dependencies (requirements.txt)

```
fastapi>=0.115
uvicorn[standard]
pydantic>=2
pydantic-settings
httpx          # async HTTP to Telnyx / Resemble / LLM
supabase       # supabase-py (DB + storage, service role)
python-jose[cryptography] # or PyJWT: verify Supabase JWTs
python-multipart # file uploads
apscheduler>=3.10
telnyx         # optional SDK; raw httpx is fine too
pynacl         # verify Telnyx webhook signatures (ed25519)
openai / anthropic # whichever LLM you choose
pydub         # audio conversion (needs ffmpeg)
tenacity       # retries on provider calls
pytest, pytest-asyncio
```

Environment variables (.env.example)

```
APP_ENV=dev
PUBLIC_BASE_URL=https://<your-ngrok-or-render-url> # used for Telnyx webhooks
FRONTEND_ORIGIN=http://localhost:5173
DEMO_MODE=true

SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
SUPABASE_JWT_SECRET= # or use JWKS URL for asymmetric keys

TELNYX_API_KEY=
TELNYX_PUBLIC_KEY= # webhook signature verification
TELNYX_CONNECTION_ID= # Call Control app / connection
TELNYX_FROM_NUMBER=+1...
TELNYX_MESSAGING_PROFILE_ID=
TELNYX_HUGH_ASSISTANT_ID=

RESEMBLE_API_KEY=
RESEMBLE_PROJECT_UUID=

LLM_PROVIDER=openai # or anthropic / telnyx
LLM_API_KEY=
LLM_MODEL=

DETECT_FAKE_THRESHOLD=0.5
SPEAKER_MATCH_THRESHOLD=0.75
```

Auth pattern

The frontend logs in with Supabase JS and sends the access token as `Authorization: Bearer <jwt>`. A FastAPI dependency verifies the token, extracts sub (user id), and every query checks the user belongs to the Circle.

```
# app/deps.py
async def get_current_user(authorization: str = Header(...)) -> User:
    token = authorization.removeprefix("Bearer ")
    claims = jwt.decode(token, settings.SUPABASE_JWT_SECRET,
                        algorithms=["HS256"], audience="authenticated")
    return User(id=claims["sub"], email=claims.get("email"))

async def require_circle_member(circle_id: UUID, user=Depends(get_current_user)):
    if not await repo.is_member(circle_id, user.id):
        raise HTTPException(403, "Not a member of this circle")
    return user
```

6. Data model

Postgres via Supabase. Put this in migrations/001_init.sql and run it in the Supabase SQL editor.

```
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text, phone text, created_at timestamptz default now()
);

create table circles (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  owner_id uuid references profiles(id),
  created_at timestamptz default now()
);

create type member_role as enum ('senior','trusted_contact');
create type enroll_status as enum ('none','processing','enrolled','failed');

create table circle_members (
  id uuid primary key default gen_random_uuid(),
  circle_id uuid references circles(id) on delete cascade,
  user_id uuid references profiles(id),          -- null for seniors (no login)
  role member_role not null,
  display_name text not null,                   -- "Grandma Ada", "Alex"
  relationship text,                            -- "grandson"
  phone_e164 text,
  timezone text default 'Africa/Lagos',
  companion_call_time time,                    -- seniors only
  companion_enabled boolean default false,
  personal_facts jsonb default '[]',           -- for Hard scenarios, family-provided
  created_at timestamptz default now()
);

create table voice_profiles (
  id uuid primary key default gen_random_uuid(),
  member_id uuid references circle_members(id) on delete cascade,
  status enroll_status default 'processing',
  sample_path text,                            -- storage path
  resemble_voice_uuid text,                   -- clone
  resemble_identity_id text,                  -- verification reference
  consent_text_version text not null,
  consent_at timestamptz not null,
  error text, created_at timestamptz default now()
);

create table practice_calls (
  id uuid primary key default gen_random_uuid(),
  circle_id uuid references circles(id) on delete cascade,
  senior_member_id uuid references circle_members(id),
  voice_member_id uuid references circle_members(id),
  scenario text, difficulty text check (difficulty in ('easy','medium','hard')),
  status text default 'queued', -- queued|dialing|in_progress|completed|failed|no_answer
  telnyx_call_control_id text,
  outcome text, score int, transcript jsonb default '[]',
  duration_seconds int, started_at timestamptz, ended_at timestamptz,
  created_by uuid references profiles(id), created_at timestamptz default now()
);
```

```

create table detection_checks (
  id uuid primary key default gen_random_uuid(),
  circle_id uuid references circles(id) on delete cascade,
  claimed_member_id uuid references circle_members(id),
  audio_path text, status text default 'processing',
  synthetic_score numeric, speaker_match_score numeric,
  verdict text,          -- real|fake|not_them|unsure
  explanation text, created_by uuid, created_at timestamptz default now()
);

create table companion_calls (
  id uuid primary key default gen_random_uuid(),
  circle_id uuid references circles(id) on delete cascade,
  senior_member_id uuid references circle_members(id),
  status text default 'queued', telnyx_call_control_id text,
  transcript jsonb default '[]', summary text,
  metrics jsonb,          -- {wpm, latency_ms, filler_rate, recall, mood}
  flags jsonb default '[]',
  duration_seconds int, started_at timestamptz, ended_at timestamptz,
  created_at timestamptz default now()
);

create table alerts (
  id uuid primary key default gen_random_uuid(),
  circle_id uuid references circles(id) on delete cascade,
  type text,              -- scam_risk|wellbeing_change|detection_fake
  severity text,          -- info|warning|urgent
  title text, message text, source_id uuid,
  acknowledged_at timestamptz, created_at timestamptz default now()
);

create table audit_log (
  id bigserial primary key, circle_id uuid, actor_id uuid,
  action text, details jsonb, created_at timestamptz default now()
);

```

Storage buckets (private): voice-samples, call-audio (generated clips), detection-uploads. Serve files only through short-lived signed URLs created by FastAPI.

7. API contract (frontend to backend)

Base path `/api/v1`. JSON everywhere except file uploads (multipart). All routes except webhooks and health require a Bearer token. FastAPI auto-generates OpenAPI at `/docs` and `/openapi.json`; give that file to Claude Code to generate a typed client.

Endpoints

Method + path	Purpose	Body / response (summary)
GET <code>/health</code>	Liveness	<code>{status:"ok"}</code>
GET <code>/me</code>	Current user + circles	<code>{id, email, circles:[{id,name,role}]}</code>
POST <code>/circles</code>	Create circle	<code>{name}</code> -> Circle
GET <code>/circles/{id}</code>	Circle overview	Circle + members + latest stats
POST <code>/circles/{id}/members</code>	Add senior or trusted contact	<code>{role, display_name, relationship, phone_e164, timezone}</code>
PATCH <code>/members/{id}</code>	Edit member, companion settings, personal facts	partial Member
DELETE <code>/members/{id}</code>	Remove member	204

Method + path	Purpose	Body / response (summary)
GET /voice/prompts	Phrases to read when recording	{prompts:[...]}
POST /members/{id}/voice	Upload sample + consent (multipart)	file, consent=true, consent_version -> {voice_profile_id, status}
GET /members/{id}/voice	Enrollment status	{status, error?}
DELETE /members/{id}/voice	Delete voice + provider data	204
GET /practice/scenarios	List scenarios	[[key, title, description, difficulties]]
POST /circles/{id}/practice-calls	Start or schedule practice call	{senior_member_id, voice_member_id, scenario, difficulty, scheduled_for?}
GET /circles/{id}/practice-calls	History	[PracticeCall]
GET /practice-calls/{id}	Status, transcript, outcome	PracticeCall
POST /practice-calls/{id}/cancel	Hang up / cancel	PracticeCall
POST /circles/{id}/detections	Upload suspicious clip (multipart)	file, claimed_member_id -> {id, status}
GET /detections/{id}	Verdict	{status, verdict, synthetic_score, speaker_match_score, explanation}
GET /circles/{id}/detections	History	[Detection]
POST /members/{id}/companion-calls	Call Hugh now (demo)	{id, status}
GET /members/{id}/companion-calls	History	[CompanionCall]
GET /members/{id}/companion-trends?days=14	Metric series + baseline	{series:{wpm:[...],latency_ms:[...]}, baseline:{...}}
GET /circles/{id}/alerts	Alerts	[Alert]
POST /alerts/{id}/ack	Acknowledge	Alert
POST /webhooks/telnyx	Telnyx call + SMS events (signature-verified)	200 fast; work in background
POST /demo/seed /demo/reset	Load or wipe demo data (DEMO_MODE only)	{ok:true}

Core response shapes

```
PracticeCall {
  id, circle_id, senior_member_id, voice_member_id,
  scenario, difficulty: "easy"|"medium"|"hard",
  status: "queued"|"dialing"|"in_progress"|"completed"|"failed"|"no_answer",
  outcome?: "hung_up_early"|"verified"|"refused"|"hesitated"|"complied",
  score?: number, duration_seconds?: number,
  transcript: [{speaker: "caller"|"senior", text, t_ms}],
  created_at, started_at?, ended_at?
}

Detection {
  id, status: "processing"|"done"|"failed",
  verdict?: "real"|"fake"|"not_them"|"unsure",
  synthetic_score?: number, speaker_match_score?: number,
  explanation?: string, claimed_member: {id, display_name}
}

Error { error: { code: string, message: string } } // all 4xx/5xx
```

8. Call flows and webhooks

Practice call state machine

```
queued --dial()--> dialing --call.answered--> in_progress
in_progress loop:
  play(opening clip)          --call.playback.ended--> start transcription
  call.transcription (final)  --> llm_service.next_turn(state, senior_text)
                                --> resemble_service.synthesize(line, voice)
                                --> storage upload --> telnyx play(url)
  guards: digits detected / max 4 min / max 8 turns --> interrupt + debrief
  call.hangup --> completed --> classify_outcome(transcript) --> score
                                --> send debrief SMS --> create alert if outcome in (hesitated, complied)
dialing --no answer 30s / call.hangup--> no_answer
```

Webhook handling rules

- Verify every Telnyx webhook signature (ed25519 with TELNYX_PUBLIC_KEY) before processing.
- Return 200 within 1 second; push work to BackgroundTasks.
- Map events to rows using client_state (base64 JSON with {kind:"practice"|"companion", id}) set when dialing.
- Make handlers idempotent: store processed event IDs and skip duplicates.
- Events to handle: call.initiated, call.answered, call.playback.ended, call.transcription, call.hangup, message.received (optional replies to SMS).

Companion call flow

```
scheduler tick (every minute) -> seniors whose local time == companion_call_time
-> build context: name, last 3 summaries, one recall question
-> start Telnyx AI Assistant call with context (or own turn loop)
call.hangup -> fetch transcript -> llm summary + recall grade + mood
-> metrics_service.compute(transcript) -> compare to baseline
-> flags -> alert_service.notify(circle, flags) if any
```

Latency budget for scam turns

Step	Target
Final transcription event after senior stops	< 1.0 s
LLM next line (short, max 40 words, streaming off)	< 1.5 s
Resemble synthesis of one sentence	< 1.5 s
Upload + playback start	< 0.5 s
Total gap	~3–4 s (play a short filler clip like "Grandma? ... are you there?" if over 3 s)

9. Frontend spec for Claude Code

The frontend is a single-page React app. It never holds provider keys; it only knows `VITE_API_BASE_URL`, `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`.

Stack

- Vite + React 18 + TypeScript, React Router, TanStack Query for API calls and polling.
- Tailwind CSS + shadcn/ui components, Recharts for trends.
- Supabase JS for login only; an `ApiClient` attaches the access token to every FastAPI request.
- Types generated from FastAPI's `/openapi.json` (`openapi-typescript`).

Pages

Route	Page	Key elements
<code>/login</code>	Sign in	Magic link email, large single field.
<code>/onboarding</code>	Create Circle wizard	Step 1 circle name, Step 2 add senior, Step 3 record your voice, Step 4 done.
<code>/c/:id</code>	Circle home	Senior card, status tiles (voices enrolled, last practice score, last Hugh call, open alerts).
<code>/c/:id/voices</code>	Voice Circle	Members list, record button with live level meter, prompts, consent, status badge.
<code>/c/:id/training</code>	Scam Training	Pick voice, scenario, difficulty (3 big buttons), Call now. Live status, then outcome card + transcript.
<code>/c/:id/check</code>	Check a call	Big upload zone, "who did they say they were?", result screen in huge text with color + icon + words.
<code>/c/:id/hugh</code>	Companion	Call history, 14-day trend chart with baseline band, call summaries, Call Hugh now.
<code>/c/:id/alerts</code>	Alerts	List with severity, plain message, Acknowledge.

Accessibility and design rules

- High contrast (WCAG AA or better), base font 18px, buttons at least 48px tall.
- One primary action per screen. Plain words: "Check a call", not "Deepfake analysis".
- Verdicts use color + icon + text together (never color alone).
- Every async action shows a status line: "Calling Grandma Ada...", "Listening...", "Done".
- Mobile-first layout; family will often use it on a phone.

Starter prompt for Claude Code

Build the VoiceCircle frontend: Vite + React + TypeScript + Tailwind + shadcn/ui, React Router, TanStack Query, Recharts, Supabase JS (auth only).

- Read ./openapi.json (exported from the FastAPI backend) and generate types with openapi-typescript into src/api/types.ts. Create src/api/client.ts that adds "Authorization: Bearer <supabase access token>" to every request to VITE_API_BASE_URL/api/v1.
- Implement routes: /login, /onboarding, /c/:id, /c/:id/voices, /c/:id/training, /c/:id/check, /c/:id/hugh, /c/:id/alerts (see PRD section 9).
- Voice recording: MediaRecorder (audio/webm), show 3-5 prompts from GET /voice/prompts, a live input level meter, re-record, consent checkbox, then POST multipart to /members/{id}/voice and poll status every 2s.
- Practice call: POST /circles/{id}/practice-calls, then poll GET /practice-calls/{id} every 2s until status is completed/failed/no_answer; render outcome card (score, outcome label in plain words) and transcript.
- Detection: drag-and-drop upload, poll GET /detections/{id}; show verdict full-screen: REAL (green, check icon), FAKE (red, warning icon), NOT THEM, NOT SURE, always with "Hang up and call back on a number you know".
- Companion: Recharts line chart of wpm and latency over 14 days with a shaded baseline band; list of call summaries.
- Accessibility: 18px base font, 48px min touch targets, AA contrast, never use color alone. Mobile-first. Clean, calm, low clutter.
- Handle errors in the shape {error:{code,message}} with friendly toasts.

10. Security, privacy and ethics

Area	Requirement
Consent	Voice enrollment requires explicit consent by the person whose voice it is, recorded with timestamp and text version. Seniors are told during onboarding that practice calls will happen.
Clone misuse	Cloned voices can only be used inside their own Circle and only for practice calls placed to that Circle's senior. No free-text TTS endpoint is exposed.
Practice safety	Practice calls never collect real financial data. Digit sequences trigger an immediate interruption and debrief. Every practice call ends with a disclosure.
Access control	Every endpoint checks Circle membership. Supabase RLS enabled as a second layer. Storage buckets private; signed URLs expire in 10 minutes.
Secrets	Provider keys live only on the backend. Webhooks signature-verified. CORS limited to the frontend origin.
Data retention	Raw detection uploads auto-delete after 30 days (P1). "Delete my voice" removes provider voices and files.
Audit	Log voice creation, every clone use, every detection, every alert in audit_log.
Health claims	Companion insights are wellness signals, not diagnoses; wording reviewed accordingly.
Regulation note	Automated/AI-voice calls are regulated in many countries. For the hackathon, call only consenting team members and test numbers.

11. Build plan (48 hours)

Times in WAT (Lagos). Deadline is Saturday Oct 10, 5:00 PM WAT. Aim to freeze features by Saturday 6:00 AM.

When	Backend (you, FastAPI)	Frontend (Claude Code)
Thu Oct 8, AM	Repo + FastAPI skeleton, config, /health. Supabase project, run 001_init.sql, buckets. JWT dependency. Deploy to Render/Railway or run with ngrok.	Scaffold Vite app, Supabase login, routing, layout, API client.
Thu Oct 8, PM	Circles + members CRUD. Voice upload + Resemble enrollment service. Export openapi.json.	Onboarding wizard, Voice Circle page with recorder.
Thu Oct 8, night	Telnyx: buy number, Call Control app, webhook endpoint with signature check. Outbound call that plays one cloned clip. Milestone: phone rings in Alex's voice.	Training page UI with polling against stub data.
Fri Oct 9, AM	Practice turn loop (transcription, LLM, synth, play). Easy + Hard prompts. Outcome classification, debrief SMS.	Wire Training to real API, outcome card, transcript.
Fri Oct 9, PM	Detection endpoint: storage + Resemble detect + verify, verdict logic.	Check-a-call upload + verdict screen.
Fri Oct 9, night	Companion: Telnyx AI Assistant with Hugh prompt, transcript fetch, metrics, alerts. Demo seed endpoint (7 days of history).	Hugh page with trend chart, Alerts page, Circle home tiles.
Sat Oct 10, 6–10 AM	Bug fixes, threshold tuning, README, architecture diagram.	Polish, accessibility pass, mobile check.
Sat Oct 10, 10 AM–2 PM	Record demo video (live calls). Screenshots.	
Sat Oct 10, by 3 PM	Submit on Devpost (2 hours of buffer before 5 PM WAT).	

Cut order if behind schedule: 1) Medium difficulty, 2) scheduled practice calls, 3) Supabase Realtime, 4) live Hugh metrics (use seeded data for the trend, keep one live call). Never cut: live practice call in a cloned voice, and live deepfake check.

12. Demo script and submission

Demo video (about 3 minutes)

Time	Scene
0:00–0:20	Hook: a real-sounding voice clip, "Grandma, I'm in trouble...". Caption: this voice is fake. One stat-free sentence on the problem.
0:20–0:50	Voice Circle: Alex records his sample, consent, "Enrolled".
0:50–1:50	Scam Training: Easy call, Grandma hangs up (score 100). Hard call, she starts to comply, call interrupts, debrief SMS arrives. Dashboard shows both.
1:50–2:20	Check a call: upload the Hard-call recording, verdict FAKE in large text.
2:20–2:50	Hugh: short live clip of a companion call, then the 14-day trend and a gentle alert to Alex.
2:50–3:10	Architecture slide (FastAPI, Telnyx, Resemble, Supabase) and Trust and Safety summary. Close on the name.

Submission checklist

- Devpost: title, one-liner, track (AI + Cybersecurity), description (problem, solution, how it works, challenges, what's next).
- Demo video link (YouTube unlisted or Vimeo).
- Public GitHub repo(s): `voicecircle-api` and `voicecircle-web`, each with README, `.env.example`, run steps.
- Architecture diagram and 4–6 screenshots.
- Trust and Safety section copied from Section 10.
- State clearly that everything was built during the hackathon window.

13. Risks and open questions

Risk	Impact	Mitigation
Telnyx number or calling to Nigeria (+234) is restricted or slow to activate	No live calls	Activate the account today; test with a US number or a team member's verified number; keep a recorded backup of the demo call.
Turn latency feels robotic	Weak Hard demo	Pre-generate openers and fillers; keep LLM lines short; Easy fully pre-generated.
Resemble clone quality from a short sample	Less convincing	Record 60 s in a quiet room; use the same mic for demo clips.
Detection thresholds misfire on phone-quality audio	Wrong verdict on stage	Tune thresholds on your own clips; return "Not sure" in the grey zone.
Webhooks unreachable locally	Calls stall	Use ngrok or deploy early; log every webhook.
Provider rate limits or credits run out	Demo breaks	Monitor usage; cache synthesized clips by text hash.

Open questions to decide today

- LLM for scam turns and classification: Telnyx-hosted model, OpenAI or Anthropic?
- Hugh on Telnyx AI Assistant (fast to build) or your own turn loop (more control)?
- Database access from FastAPI: `supabase-py` (quick) or `SQLAlchemy async + asyncpg` (typed, more setup)?
- Hosting: Render, Railway or Fly.io for the API; Vercel or Netlify for the frontend?
- Which senior phone number will be used live in the demo, and is it reachable from your Telnyx number?